End-to-End Project Report: Drowsiness Detection System

Date: April 2026
Role: Lead Project Manager

---

1. The Big Picture

What was this project? Why did we bother doing it in the first place?

We set out to solve a very real, very dangerous problem: drivers falling asleep at the wheel. The goal was to build a smart, offline, edge-computing system that watches a driver's eyes in real-time and sounds a blaring alarm if they start dozing off.

We specifically targeted the **Raspberry Pi 4B** so the final product could be cheap, highly portable, and run entirely without an internet connection. We didn't want to rely on cloud processing or Wi-Fi on a lonely highway at 3 AM. It needed to be fast, local, and 100% reliable.

---

2. The "Small Stuff" Breakdown

Here are the tiny moving parts that people usually ignore but actually make the project tick:

- **File Structure & Naming:** We kept the architecture modular and clean.
  - `core_engine.py`: The "brain". This holds all the complex math, hardware logic, and timing algorithms.
  - `drowsiness_detector.py`: The "eyes and voice". This handles the camera feed, draws the visual UI overlay, and runs the main loop.
  - `drowsiness_detector_config.json`: The "control panel". This lets us tweak thresholds (like how many frames trigger a warning) without having to touch a single line of code.
  - `test_functions.py`: The "safety net". 32 automated unit tests that run instantly to ensure we don't accidentally break the math when adding new features.
- **Hardware Fallbacks:** We wrote the code gracefully. If it’s running on a regular Windows laptop during development, it realizes there are no physical pins and "fakes" the hardware buzzer (printing to the console). If it’s on the Pi, it automatically hooks into the real GPIO pins to trigger the 5V alarm.
- **Daily Syncs & Tracking:** We tracked everything via rapid iteration loops, keeping the codebase strictly separated into logical chunks so we could seamlessly swap out bad components without breaking the whole system.

---

3. The Timeline

1. **The Lightbulb Moment:** The need for a local, low-latency drowsiness detector is identified and scoped for Raspberry Pi.
2. **Phase 1: The Basics (Haar Cascades):** We started with standard OpenCV Haar Cascades to detect faces and eyes. It was incredibly fast, but unfortunately, it was also incredibly jumpy and unreliable.
3. **Phase 2: The "Aha" Math (Degradation Index):** We realized that waiting for the eyes to fully close was waiting too long. We invented a novel **Blink Quality Degradation Index** to measure how "sluggish" blinks were getting compared to the user's normal baseline. This let us predict sleep 3-5 seconds *before* it happened.
4. **Phase 3: The Wall (Haar Sucks):** The Haar cascades kept flickering. It would lose the eyes for a split second due to bad lighting, and the system would think the driver was asleep, blasting a false alarm. We tried adding smoothers and memory loops, but it was just putting a band-aid on a broken leg.
5. **Phase 4: The Pivot (MediaPipe):** We scrapped Haar cascades and upgraded to Google's MediaPipe Face Mesh. Suddenly, we had 468 3D points locked onto the face perfectly. No more flickering. We updated our EAR (Eye Aspect Ratio) math to use precise 3D Euclidean distances.
6. **The Finish Line:** All 32 automated tests passed, the UI looked beautiful, and the buzzer logic was completely stable.

---

4. The Roadblocks

What tripped us up? How did we fix it?

- **Roadblock 1: The Jumpy Eye Problem.** As mentioned, Haar Cascades were awful in uneven lighting. A slight head turn or shadow triggered a false alarm.
    - *The Fix:* We migrated to MediaPipe Face Mesh. It gives sub-millimeter precision for eye tracking, completely eliminating the false positive issue.
- **Roadblock 2: The Dlib Disaster.** We initially tried using a popular library called `dlib` to fix the jumpy eyes (as it is the standard academic approach). It completely failed to install on Windows because it requires heavy C++ compilers, and we knew it would run terribly slow (~2-5 FPS) on the Raspberry Pi anyway.
    - *The Fix:* Pivoted to MediaPipe again. It’s pure Python, installs instantly via pip, and runs at a buttery 15-30 FPS on the Pi using CPU optimization.
- **Roadblock 3: The "Already Asleep" Problem.** Standard formulas only trigger an alarm when eyes are fully shut. By then, the car is already off the road.
    - *The Fix:* Our custom "Blink Quality Degradation" algorithm. It auto-calibrates to the driver's normal blink speed (the first 5 blinks of the session) and catches them *slowing down* before they actually fall asleep.

---

5. The Win

What does success look like now that we’re done?

We have a production-ready, highly robust Python application. You can sit in front of the camera, talk, move your head, and wear glasses, and it won't freak out.

But the second you start doing slow, heavy "tired blinks," the screen flashes yellow (Warning). Close your eyes for a second, and it hits critical red, immediately sounding the hardware alarm. It runs offline, respects user privacy (no video is saved or sent to the cloud), and is perfectly optimized for cheap hardware. We absolutely nailed it.

---

6. Glossary for Humans

- **EAR (Eye Aspect Ratio):** A math trick to measure how open an eye is. Imagine measuring the height of your eye divided by the width. If that number drops to near zero, the eye is closed.
- **MediaPipe:** A powerful tool made by Google that basically draws a hyper-accurate 3D web over a face in a live video feed. It lets us know exactly where the eyelids are.
- **Haar Cascades:** An older, simpler way to find faces in images. It’s fast but gets confused easily (like thinking a shadow is an eye). We ditched this.
- **Raspberry Pi (RPi 4B):** A full computer the size of a credit card. It’s cheap and great for DIY electronics but isn't super powerful, so code has to be highly optimized to run smoothly.
- **GPIO Pins:** The little metal spikes on a Raspberry Pi that let you plug in physical hardware, like LEDs or buzzers.
- **Unit Tests:** Small automated scripts that check our code every time we make a change, ensuring we didn't accidentally break the math or the logic.